

Software Requirements Specification (SRS)

Dynamic Capacity Broker • Adaptive Token Bucket • Strict FCFS Engine

PLATFORM NAME	INGRESS BALANCER	ALLOCATION ENGINE	DURABILITY STORE
TicketWala	Adaptive Token Bucket (Lua)	Strict FCFS FIFO Queue (RPOP)	PostgreSQL 18 Ledger (Streams)

1. Architectural Topology & Adaptive Ingress Flow

TicketWala decouples high-velocity write traffic from relational storage by combining an in-memory transactional inventory broker with an **Adaptive Ingress Balancer**. Ingress rates automatically calibrate to remaining inventory scarcity, eliminating database deadlocks and connection exhaustion while guaranteeing sub-second holding locks.

Adaptive Request Balancing Pipeline

```
[ Next.js 15 Client ] <--- WebSocket / HTTP Polling ---> [ Operations Observatory ]
|
HTTP POST /hold (Burst of 5,000+ Requests)
v
[ Fastify TypeScript API Gateway ]
- Zod Payload Validation
- Scoped Idempotency Check (SHA-256)
|
1. Adaptive Token Bucket Admission (adaptive_balance.lua)
- Computes: remaining_seats = LLEN(available_queue)
- If remaining_seats == 0: Instant O(1) Short-Circuit (409 SOLD_OUT)
- If remaining_seats > 0: Dynamically scales refill rate & capacity
v
[ Redis In-Memory Transactional Broker (Single Master) ]
- 2. Strict FCFS Unit Allocation (RPOP ticketwala:inventory:available_queue)
- 3. Cryptographic Token Generation (128-bit secret)
- 4. Server-Side 120s TTL Counter & Monotonic Version Fencing (v = v + 1)
- 5. Atomic Append to Stream ('ticketwala:events')
|
Redis Stream Consumer Group ('ticketwala_workers')
v
[ Asynchronous Persistence Worker ]
- At-Least-Once Consumption (XREADGROUP)
- Idempotent Transactional Upsert (PostgreSQL)
- Crash Recovery & Reclamation (XAUTOCLAIM)
|
[ PostgreSQL 18 Relational Ledger ] <--- Cross-Store Audit --- [ Invariant Auditor ]
- reservations Table
- reservation_events Immutable Log
- idempotency_records Cache
```

2. Core Algorithmic Design: Redis Lua Scripts

2.1 Adaptive Request Balancing Script (`adaptive\_balance.lua`)

This script balances incoming request throughput automatically against available seat inventory, protecting system resources from thrashing:

```
-- KEYS: [1] queue_key, [2] bucket_key
-- ARGV: [1] base_capacity, [2] base_refill_rate, [3] now_ms
local avail_seats = redis.call('LLEN', KEYS[1])

-- FAST SHORT-CIRCUIT: Zero seats remaining
if avail_seats == 0 then
    return cjson.encode({ admitted = 0, reason = "SOLD_OUT", code = 409 })
end

-- ADAPTIVE BALANCING: Scale token bucket dynamically as a function of remaining seats
-- High inventory (>50%): High throughput burst allowed
-- Low inventory (<10%): Progressive throttle into serialized FIFO line
local bucket = redis.call('HMGET', KEYS[2], 'tokens', 'last_updated')
local base_cap = tonumber(ARGV[1])
local base_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

-- Dynamic capacity scaling
local dynamic_cap = math.min(base_cap, math.max(10, avail_seats * 2))
local dynamic_rate = math.min(base_rate, math.max(5, avail_seats))

local tokens = tonumber(bucket[1]) or dynamic_cap
local last_updated = tonumber(bucket[2]) or now

local elapsed = math.max(0, (now - last_updated) / 1000)
tokens = math.min(dynamic_cap, tokens + (elapsed * dynamic_rate))
last_updated = now

if tokens >= 1 then
    tokens = tokens - 1
    redis.call('HMSET', KEYS[2], 'tokens', tokens, 'last_updated', last_updated)
    return cjson.encode({ admitted = 1, remaining_seats = avail_seats, code = 200 })
else
    redis.call('HMSET', KEYS[2], 'tokens', tokens, 'last_updated', last_updated)
    return cjson.encode({ admitted = 0, reason = "RATE_LIMITED", code = 429 })
end
```

2.2 Strict First-Come-First-Served Atomic Hold (`hold\_fcfs.lua`)

Pops the next available unit strictly from the FIFO List queue in arrival order, binds the reservation, sets the 120s TTL counter, and appends the stream event atomically:

```

-- KEYS: [1] fifo_queue, [2] unit_prefix, [3] res_prefix, [4] idemp_prefix, [5] stream_key
-- ARGV: [1] idemp_scope_key, [2] req_fingerprint, [3] res_id, [4] token_hash, [5] ttl_seconds, [6] server_now
local idemp_key = KEYS[4] .. ":" .. ARGV[1]
local cached_res = redis.call('GET', idemp_key)
if cached_res then return cached_res end -- Deterministic Idempotency

-- 1. Strict FCFS: Pop next unit from FIFO List in exact arrival sequence
local unit_id = redis.call('RPOP', KEYS[1])
if not unit_id then
    return cjson.encode({ error = "SOLD_OUT", code = 409 })
end

-- 2. Bind unit and reservation atomically with monotonic versioning
local unit_key = KEYS[2] .. ":" .. unit_id
local res_key = KEYS[3] .. ":" .. ARGV[3]
local expires_at = tonumber(ARGV[6]) + tonumber(ARGV[5])
local version = 1

redis.call('HSET', unit_key, 'status', 'HELD', 'reservation_id', ARGV[3],
    'token_hash', ARGV[4], 'expires_at', expires_at, 'version', version)
redis.call('HSET', res_key, 'unit_id', unit_id, 'status', 'HELD',
    'token_hash', ARGV[4], 'expires_at', expires_at, 'version', version)

-- 3. Append to event stream for eventual consistency
local event_id = redis.call('XADD', KEYS[5], '*',
    'event_type', 'HOLD_CREATED', 'reservation_id', ARGV[3], 'unit_id', unit_id,
    'version', version, 'occurred_at', ARGV[6])

-- 4. Cache idempotency outcome (10 min TTL)
local response = cjson.encode({
    reservation_id = ARGV[3], unit_id = unit_id, status = 'HELD',
    expires_at = expires_at, version = version, event_id = event_id
})
redis.call('SETEX', idemp_key, 600, response)
return response

```

2.3 Instant Abandonment & Timeout Release Script ('release_fcfs.lua')

When a checkout is abandoned or times out, the unit is pushed to the head of the FIFO list via `LPUSH` within < 5ms, making it immediately available for the next awaiting customer:

```

-- KEYS: [1] fifo_queue, [2] unit_prefix, [3] res_prefix, [4] stream_key
-- ARGV: [1] res_id, [2] unit_id, [3] expected_version, [4] server_now
local unit_key = KEYS[2] .. ":" .. ARGV[2]
local current_res = redis.call('HGET', unit_key, 'reservation_id')
local current_ver = tonumber(redis.call('HGET', unit_key, 'version'))

-- Monotonic Version Fence: Prevent stale releases from revoking newer owners
if current_res ~= ARGV[1] or current_ver ~= tonumber(ARGV[3]) then
    return cjson.encode({ status = "IGNORED_STALE_RELEASE", code = 200 })
end

-- Return unit to head of FIFO queue for immediate re-allocation
redis.call('HDEL', unit_key, 'reservation_id', 'token_hash', 'expires_at')
redis.call('HSET', unit_key, 'status', 'AVAILABLE')
redis.call('HINCRBY', unit_key, 'version', 1)
redis.call('LPUSH', KEYS[1], ARGV[2]) -- Instant availability for next FCFS requester

local res_key = KEYS[3] .. ":" .. ARGV[1]
redis.call('HSET', res_key, 'status', 'RELEASED')

redis.call('XADD', KEYS[4], '*', 'event_type', 'HOLD_RELEASED',
    'reservation_id', ARGV[1], 'unit_id', ARGV[2], 'version', current_ver + 1, 'occurred_at', ARGV[4])
return cjson.encode({ status = "RELEASED", unit_id = ARGV[2], code = 200 })

```

3. Relational Storage Schema & Eventual Consistency

Redis Key Family Hierarchy

- `ticketwala:inventory:available_queue`
→ FIFO List of available unit IDs
(`["unit-001", ...]`)
- `ticketwala:token_bucket` → Hash
(`tokens`, `last_updated`)
- `ticketwala:unit:{unitId}` → Hash
(`status`, `reservation_id`, `token_hash`, `expires_at`, `version`)
- `ticketwala:reservation:{resId}` → Hash
(`unit_id`, `status`, `token_hash`, `expires_at`, `confirmed_at`, `version`)
- `ticketwala:idempotency:{scope}:{key}`
→ String (Cached JSON outcome)
- `ticketwala:events` → Redis Stream
(Append-only immutable event log)

PostgreSQL Relational Schema (DDL)

```
CREATE TABLE reservations (
  reservation_id UUID PRIMARY KEY,
  unit_id VARCHAR(32) NOT NULL,
  status VARCHAR(20) NOT NULL, -- HELD, CONFIRMED, EXPIRED, RELEASED
  token_hash VARCHAR(64) NOT NULL,
  version INT NOT NULL DEFAULT 1,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at TIMESTAMPTZ NOT NULL,
  confirmed_at TIMESTAMPTZ,
  CONSTRAINT check_status CHECK (status IN ('HELD', 'CONFIRMED', 'EXPIRED', 'RELEASED'))
);

CREATE TABLE reservation_events (
  event_id VARCHAR(64) PRIMARY KEY, -- Redis Stream ID
  reservation_id UUID NOT NULL REFERENCES reservations(reservation_id),
  unit_id VARCHAR(32) NOT NULL,
  event_type VARCHAR(32) NOT NULL,
  version INT NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL,
  processed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE idempotency_records (
  scope VARCHAR(64) NOT NULL,
  idempotency_key VARCHAR(128) NOT NULL,
  request_fingerprint VARCHAR(64) NOT NULL,
  outcome JSONB NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (scope, idempotency_key)
);
```

4. Asynchronous Persistence Worker & Eventual Consistency

The worker runs as an independent TypeScript process consuming events from Redis Stream `ticketwala:events` under consumer group `ticketwalla_workers`.

The At-Least-Once Delivery & Idempotent Upsert Guarantee

1. Fetches pending or new events via `XREADGROUP GROUP ticketwalla_workers worker-1 COUNT 50 BLOCK 2000 STREAMS ticketwala:events >`
2. Executes an idempotent SQL upsert in PostgreSQL:

```
INSERT INTO reservations (reservation_id, unit_id, status, token_hash, version, expires_at)
VALUES ($1, $2, $3, $4, $5, $6)
ON CONFLICT (reservation_id) DO UPDATE SET
  status = EXCLUDED.status,
  version = EXCLUDED.version,
  confirmed_at = CASE WHEN EXCLUDED.status = 'CONFIRMED' THEN NOW() ELSE reservations.confirmed_at END
WHERE EXCLUDED.version > reservations.version; -- Fencing against out-of-order replay
```

3. Appends to `reservation_events` table using `ON CONFLICT (event_id) DO NOTHING`.
4. Acknowledges the message in Redis via `XACK ticketwala:events ticketwalla_workers $eventId`.

5. Verification Against Headless Load-Testing Suite (k6)

TicketWala includes a headless k6 load-testing suite specifically tailored to verify adaptive balancing under contention:

- **Configurable Workload Matrix:** Tests 5,000+ requests across parameterized seat capacities (50, 200, 1,000 units).
- **Assertion 1 (Zero Double-Bookings):** Total HTTP 201 Created responses exactly equal available inventory; remaining requests receive clean HTTP 409 Sold Out responses.
- **Assertion 2 (Adaptive Balancing):** Ingress token bucket dynamically regulates traffic without memory or socket saturation.

- **Assertion 3 (Strict FCFS Ordering):** Allocated units match the monotonic timestamp sequence of request arrival.
- **Assertion 4 (Sub-Second Latency):** Hold acquisition $p_{50} < 10\text{ms}$, $p_{95} < 50\text{ms}$, $p_{99} < 100\text{ms}$.
- **Assertion 5 (Instant Release):** Checkouts abandoned midway trigger instant release, verified by immediate subsequent allocation to next queued request.